

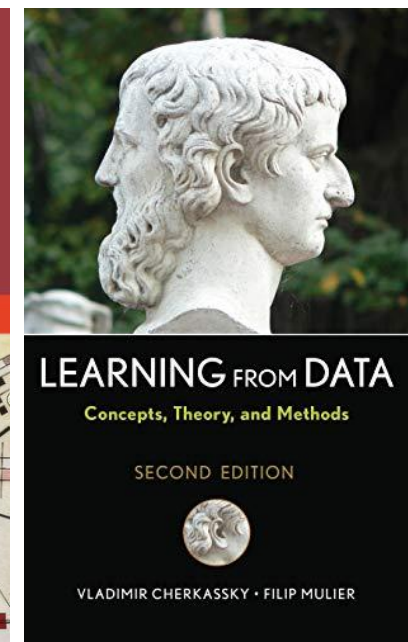
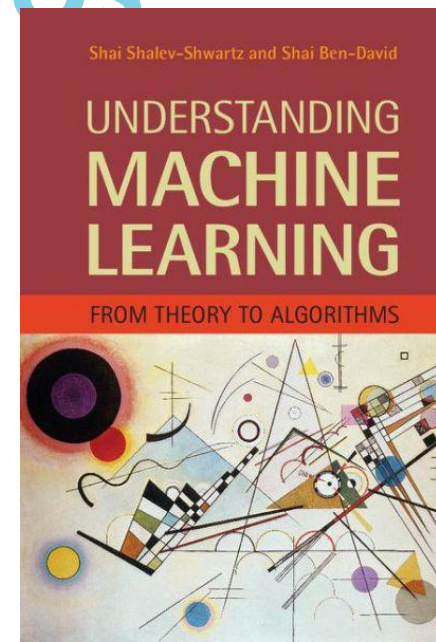
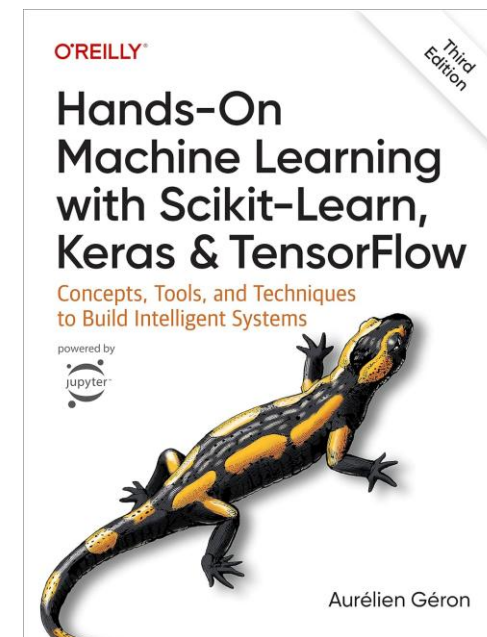
# Aprendizaje Automático

## Curso 2025-2026

### Tema 5. Aprendizaje Profundo. Fundamentos



#### Bibliografía



- 
- Tema 1: Conceptos básicos del Aprendizaje Automático
  - Tema 2: Aprendizaje Supervisado. Regresión y Clasificación. Modelos. Evaluación
  - Tema 3: Ingeniería de Características y Preprocesamiento de Datos. Calidad de Datos
  - Tema 4: Aprendizaje no Supervisado
  - **Tema 5: Aprendizaje Profundo. Fundamentos**
  - Tema 6: Aprendizaje Profundo en diferentes tipos de datos y problemas
  - Tema 7. Aprendizaje por refuerzo
  - Tema 8: Optimización en Aprendizaje Automático. Hiperparámetros, selección de modelos y regularización
  - Tema 9: Aspectos Avanzados, retos y desafíos del aprendizaje automático

# Entrenamiento de redes neuronales profundas

- ¿Y si necesita abordar un problema complejo, como detectar cientos de tipos de objetos en imágenes de alta resolución? Puede que necesites entrenar una RNA mucho más profunda, quizá con 10 capas o muchas más, cada una de ellas con cientos de neuronas, enlazadas por cientos de miles de conexiones. Entrenar una red neuronal profunda no es un paseo. Estos son algunos de los problemas con los que te puedes encontrar:
  - Puede que te encuentres con el problema de que los gradientes se hagan cada vez más pequeños o más grandes, cuando fluyen hacia atrás a través de la DNN durante el entrenamiento. Ambos problemas hacen que las capas inferiores sean muy difíciles de entrenar.
  - Puede que no disponga de suficientes datos de entrenamiento para una red tan grande, o que sea demasiado costoso etiquetarla.
  - El entrenamiento puede ser extremadamente lento.
  - Un modelo con millones de parámetros correría el grave riesgo de sobreajustar el conjunto de entrenamiento, especialmente si no hay suficientes instancias de entrenamiento o si son demasiado ruidosas.

# El problema de los degradados que desaparecen y explotan

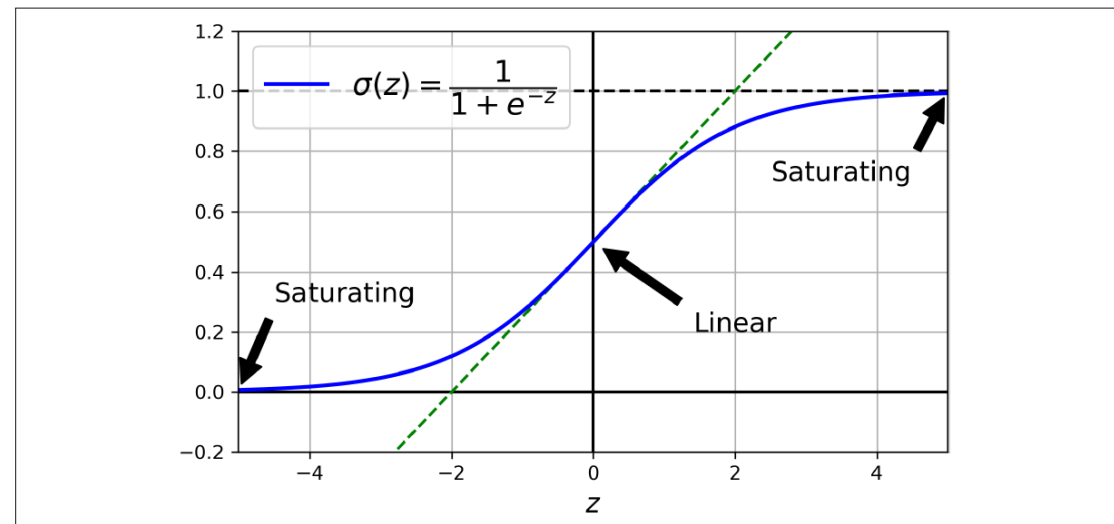
- La segunda fase del algoritmo de retropropagación funciona yendo de la capa de salida a la de entrada, propagando el gradiente de error por el camino. Una vez que el algoritmo ha calculado el gradiente de la función de coste con respecto a cada parámetro de la red, utiliza estos gradientes para actualizar cada parámetro con un paso de descenso de gradiente.
- Por desgracia, los gradientes suelen ser cada vez más pequeños a medida que el algoritmo avanza hacia las capas inferiores. Como resultado, la actualización del descenso de gradiente deja los pesos de conexión de las capas inferiores prácticamente sin cambios, y el entrenamiento nunca converge a una buena solución. Esto se conoce como el problema de los *gradientes de fuga (que se desvanecen)*.
- En algunos casos, puede ocurrir lo contrario: los gradientes pueden crecer más y más hasta que las capas obtienen actualizaciones de peso increíblemente grandes y el algoritmo diverge. Este es el problema de los *gradientes explosivos*, que aparece con más frecuencia en las redes neuronales recurrentes.
- En términos más generales, las redes neuronales profundas adolecen de gradientes inestables; las distintas capas pueden aprender a velocidades muy diferentes.

# El problema de los degradados que desaparecen y explotan

- Este desafortunado comportamiento se observó empíricamente hace mucho tiempo, y fue una de las razones por las que las redes neuronales profundas se abandonaron en su mayoría a principios de la década de 2000.
- No estaba claro qué causaba que los gradientes fueran tan inestables al entrenar una DNN, pero se arrojó algo de luz en un artículo de 2010 de Xavier Glorot y Yoshua Bengio.
- Los autores encontraron algunos sospechosos, como la combinación de la popular función de activación sigmoidea (logística) y la técnica de inicialización de pesos más popular en ese momento (es decir, una distribución normal con una media de 0 y una desviación estándar de 1).
- Demostraron que con esta función de activación y este esquema de inicialización, la varianza de las salidas de cada capa es mucho mayor que la varianza de sus entradas. Avanzando en la red, la varianza sigue aumentando después de cada capa hasta que la función de activación se satura en las capas superiores. En realidad, esta saturación empeora por el hecho de que la función sigmoidea tiene una media de 0.5, no de 0 (la función tangente hiperbólica tiene una media de 0 y se comporta ligeramente mejor que la función sigmoidea en redes profundas).

# El problema de los degradados que desaparecen y explotan

- Si se observa la función de activación sigmoidea, se puede ver que cuando las entradas son grandes (negativas o positivas), la función se satura en 0 o 1, con una derivada extremadamente cercana a 0 (es decir, la curva es plana en ambos extremos).
- Así, cuando la retropropagación se pone en marcha, prácticamente no tiene gradiente que propagar a través de la red, y el poco gradiente que existe se va diluyendo a medida que la retropropagación desciende por las capas superiores, de modo que realmente no queda nada para las capas inferiores.





# Inicialización de Glorot y He

- Glorot y Bengio proponen una forma de aliviar significativamente el problema de los gradientes inestables.
- Señalan que necesitamos que la señal fluya correctamente en ambas direcciones: en la dirección de avance al hacer predicciones, y en la dirección inversa al retropropagar gradientes. No queremos que la señal se apague, ni que explote y se sature.
- Para que la señal fluya correctamente, los autores argumentan que necesitamos que la varianza de las salidas de cada capa sea igual a la varianza de sus entradas, y necesitamos que los gradientes tengan igual varianza antes y después de fluir a través de una capa en sentido inverso.
- En realidad, no es posible garantizar ambas cosas a menos que la capa tenga el mismo número de entradas y salidas (estos números se denominan fan-in y fan-out de la capa), pero Glorot y Bengio propusieron un buen compromiso que ha demostrado funcionar muy bien en la práctica:
  - los pesos de conexión de cada capa deben inicializarse aleatoriamente como se describe aquí, donde  $fan_{avg} = (fan_{in} + fan_{out}) / 2$ . Esta es la inicialización Glorot.

Normal distribution with mean 0 and variance  $\sigma^2 = \frac{1}{fan_{avg}}$

Or a uniform distribution between  $-r$  and  $+r$ , with  $r = \sqrt{\frac{3}{fan_{avg}}}$

# Inicialización de Glorot y He

- Si se sustituye  $fan_{avg}$  por  $fan_{in}$ , se obtiene una estrategia de inicialización que Yann LeCun propuso en la década de 1990. La llamó inicialización LeCun.
- La inicialización LeCun es equivalente a la inicialización Glorot cuando  $fan_{in} = fan_{out}$ . Los investigadores tardaron más de una década en darse cuenta de la importancia de este truco. El uso de la inicialización Glorot puede acelerar el entrenamiento considerablemente, y es una de las prácticas que condujeron al éxito del aprendizaje profundo.
- La estrategia de inicialización propuesta para la función de activación ReLU y sus variantes se denomina *inicialización He* o *inicialización Kaiming*, en honor al primer autor del artículo. Para SELU, se utiliza el método de inicialización de Yann LeCun, preferiblemente con una distribución normal. En breve trataremos todas estas funciones de activación.

Table 11-1. Initialization parameters for each type of activation function

| Initialization | Activation functions                     | $\sigma^2$ (Normal) |
|----------------|------------------------------------------|---------------------|
| Glorot         | None, tanh, sigmoid, softmax             | $1 / fan_{avg}$     |
| He             | ReLU, Leaky ReLU, ELU, GELU, Swish, Mish | $2 / fan_{in}$      |
| LeCun          | SELU                                     | $1 / fan_{in}$      |

Código

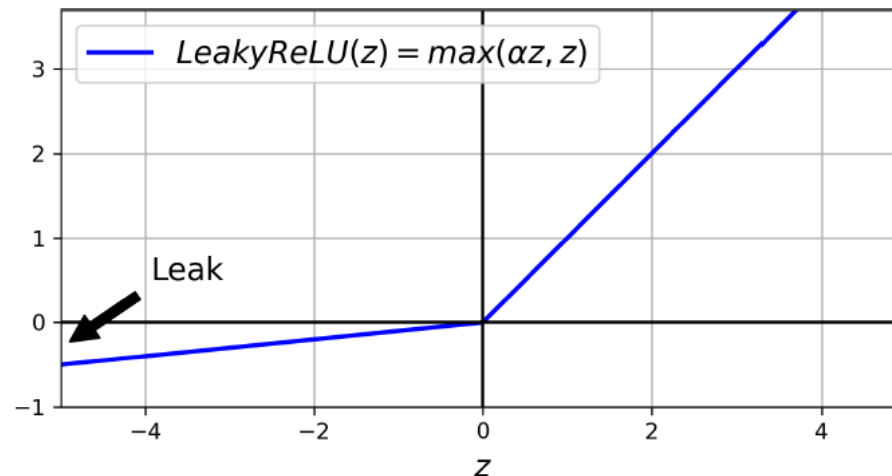


# Mejores funciones de activación

- Los gradientes inestables se debían en parte a una mala elección de la función de activación. Hasta entonces, la mayoría de la gente había dado por sentado que si la Madre Naturaleza había optado por utilizar funciones de activación aproximadamente sigmoideas en las neuronas biológicas, debían de ser una elección excelente.
- Pero otras funciones de activación se comportan mucho mejor en redes neuronales profundas: la función de activación ReLU, sobre todo porque no satura para valores positivos y también porque es muy rápida de calcular.
- Por desgracia, la función de activación ReLU no es perfecta. Sufre un problema conocido como ReLU moribundas: durante el entrenamiento, algunas neuronas "mueren", lo que significa que dejan de dar una salida distinta de 0.
- En algunos casos, es posible que la mitad de las neuronas de la red estén muertas, sobre todo si se utiliza una tasa de aprendizaje elevada. Una neurona muere cuando sus pesos se ajustan de tal forma que la entrada de la función ReLU (es decir, la suma ponderada de las entradas de la neurona más su término de sesgo) es negativa para todas las instancias del conjunto de entrenamiento.

# Leaky ReLU

- La función de activación ReLU con fugas se define como  $\text{LeakyReLU}_\alpha(z) = \max(\alpha z, z)$ . El hiperparámetro  $\alpha$  define el grado de "fuga" de la función: es la pendiente de la función para  $z < 0$ . Tener una pendiente para  $z < 0$  garantiza que las ReLU con fugas nunca mueran; pueden entrar en un coma prolongado, pero tienen la posibilidad de despertar.



# Leaky ReLU

- Un artículo de 2015 de Bing Xu et al. comparó varias variantes de la función de activación ReLU, y una de sus conclusiones fue que las variantes con fugas siempre superaban a la función de activación ReLU estricta. De hecho, establecer  $\alpha = 0.2$  (una fuga enorme) parecía dar como resultado un mejor rendimiento que  $\alpha = 0.01$  (una fuga pequeña).
- El artículo también evaluaba el ReLU *aleatorio con fugas* (RReLU), en el que  $\alpha$  se elige aleatoriamente en un rango dado durante el entrenamiento y se fija en un valor medio durante las pruebas. RReLU también funcionó bastante bien y pareció actuar como regularizador, reduciendo el riesgo de sobreajuste del conjunto de entrenamiento.
- Por último, se evaluó el ReLU *paramétrico con fugas* (PReLU), en el que se permite el aprendizaje de  $\alpha$  durante el entrenamiento: en lugar de ser un hiperparámetro, se convierte en un parámetro que puede modificarse mediante retropropagación como cualquier otro parámetro. Según los informes, PReLU supera ampliamente a ReLU en grandes conjuntos de datos de imágenes, pero en conjuntos de datos más pequeños corre el riesgo de sobreajustar el conjunto de entrenamiento.

# ELU y SELU

- ReLU, leaky ReLU y PReLU tienen el inconveniente de que no son funciones suaves: sus derivadas cambian bruscamente (en  $z = 0$ ). Este tipo de discontinuidad puede hacer que el descenso por gradiente rebote alrededor del óptimo y ralentice la convergencia.
- En 2015, un artículo de Djork-Arné Clevert et al. propuso una nueva función de activación, denominada *unidad lineal exponencial* (ELU), que superó a todas las variantes de ReLU en los experimentos de los autores: se redujo el tiempo de entrenamiento y la red neuronal obtuvo mejores resultados en el conjunto de pruebas. Esta ecuación muestra la definición de esta función de activación.

$$\text{ELU}_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

# ELU y SELU

- ▶ La función de activación ELU se parece mucho a la función ReLU, con algunas diferencias importantes:
  - ▶ Toma valores negativos cuando  $z < 0$ , lo que permite que la unidad tenga una salida media más cercana a 0 y ayuda a aliviar el problema de los gradientes de fuga. El hiperparámetro  $\alpha$  define el valor opuesto al que se aproxima la función ELU cuando  $z$  es un número negativo grande. Normalmente se establece en 1, pero se puede ajustar como cualquier otro hiperparámetro.
  - ▶ Tiene un gradiente distinto de cero para  $z < 0$ , lo que evita el problema de las neuronas muertas.
  - ▶ Si  $\alpha$  es igual a 1, entonces la función es suave en todas partes, incluso alrededor de  $z = 0$ , lo que ayuda a acelerar el descenso del gradiente, ya que no rebota tanto a la izquierda y a la derecha de  $z = 0$ .

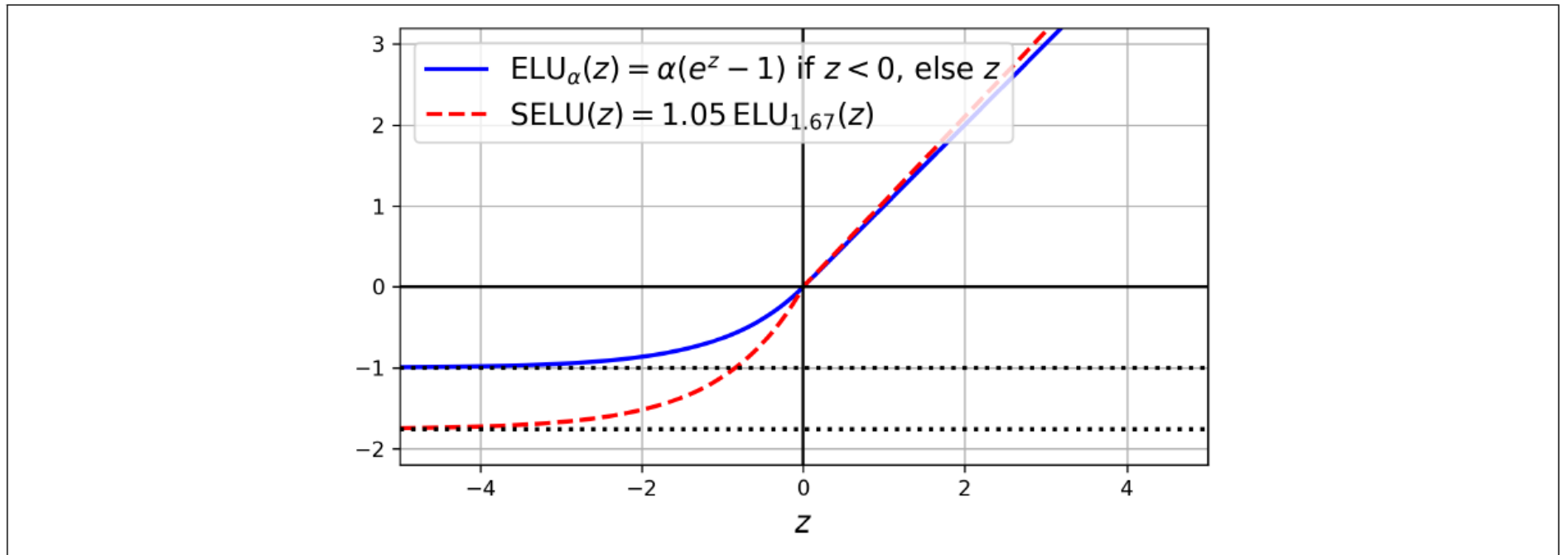
# ELU y SELU

- Un artículo de 2017 de Günter Klambauer et al. introdujo la función de activación ELU escalada (SELU).
- Es una variante escalada de la función de activación ELU (aproximadamente 1,05 veces ELU, utilizando  $\alpha \approx 1,67$ ).
- Los autores demostraron que si se construye una red neuronal compuesta exclusivamente por una pila de capas densas (es decir, una MLP), y si todas las capas ocultas utilizan la función de activación SELU, entonces la red se autonormalizará:
  - la salida de cada capa tenderá a preservar una media de 0 y una desviación estándar de 1 durante el entrenamiento, lo que resuelve el problema de los gradientes de desvanecimiento/explosión. Como resultado, la función de activación SELU puede superar a otras funciones de activación para MLP, especialmente las profundas.



# ELU y SELU

- ▶ SELU tiene importantes limitaciones, por lo que, a pesar de sus promesas, el SELU no consiguió mucha atención.

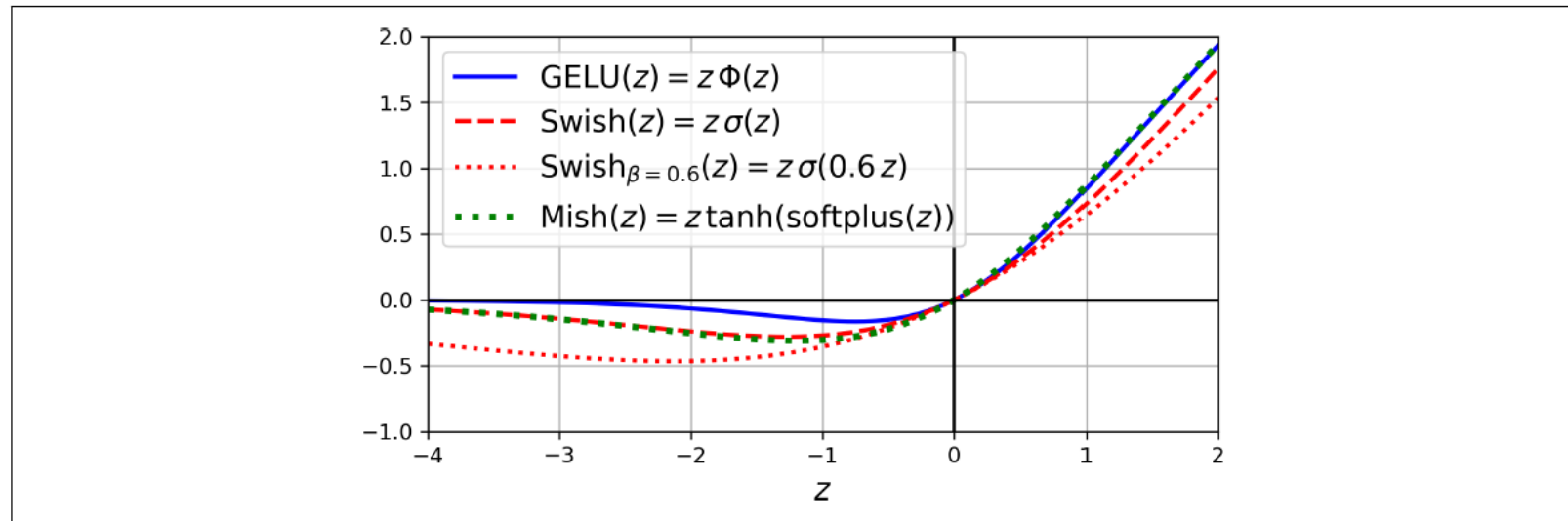


Código

# GELU y Swish

- *GELU* se introdujo en un artículo de 2016 de Dan Hendrycks y Kevin Gimpel. Una vez más, puede considerarse una variante suave de la función de activación ReLU. Su definición se da en la siguiente ecuación, donde  $\Phi$  es la función de distribución acumulativa gaussiana estándar (CDF):  $\Phi(z)$  corresponde a la probabilidad de que un valor muestreado aleatoriamente de una distribución normal de media 0 y varianza 1 sea menor que  $z$ .

$$\text{GELU}(z) = z\Phi(z)$$



Dan Hendrycks y Kevin Gimpel, "Gaussian Error Linear Units (GELUs)", arXiv preprint arXiv:1606.08415 (2016).

# GELU y Swish

- GELU se parece a ReLU: se aproxima a 0 cuando su entrada  $z$  es muy negativa, y se aproxima a  $z$  cuando  $z$  es muy positiva. Sin embargo, mientras que todas las funciones de activación que hemos analizado hasta ahora eran convexas y monótonas, la función de activación GELU no es ninguna de las dos cosas:
  - de izquierda a derecha, comienza yendo recto, luego se contonea hacia abajo, alcanza un punto bajo alrededor de  $-0,17$  (cerca de  $z \approx -0,75$ ), y finalmente rebota hacia arriba y termina yendo recto hacia la parte superior derecha.
- Esta forma bastante compleja y el hecho de que tenga una curvatura en cada punto pueden explicar por qué funciona tan bien, especialmente para tareas complejas: el descenso por gradiente puede encontrar más fácil ajustarse a patrones complejos.
- En la práctica, suele superar a todas las demás funciones de activación analizadas hasta ahora. Sin embargo, es un poco más intensiva desde el punto de vista computacional, y el aumento de rendimiento que proporciona no siempre es suficiente para justificar el coste adicional.
- Dicho esto, es posible demostrar que es aproximadamente igual a  $z\sigma(1.702 z)$ , donde  $\sigma$  es la función sigmoidea: utilizar esta aproximación también funciona muy bien, y tiene la ventaja de ser mucho más rápida de calcular.

# GELU y Swish

- El documento de GELU también introdujo la función de activación sigmoide lineal unitaria (SiLU), que es igual a  $z\sigma(z)$ , pero fue superada por GELU en las pruebas de los autores.
- Un artículo de 2017 de Prajit Ramachandran et al. redescubrió la función SiLU mediante la búsqueda automática de buenas funciones de activación. Los autores la llamaron *Swish*, y el nombre se puso de moda.
- Swish superó a todas las demás funciones, incluida GELU. Más tarde, Ramachandran et al. generalizaron Swish añadiendo un hiperparámetro adicional  $\beta$  para escalar la entrada de la función sigmoidea.
- La función Swish generalizada es  $\text{Swish}_\beta(z) = z\sigma(\beta z)$ , por lo que GELU es aproximadamente igual a la función Swish generalizada utilizando  $\beta = 1,702$ .
- Puede ajustar  $\beta$  como cualquier otro hiperparámetro. Alternativamente, también es posible hacer  $\beta$  entrenable y dejar que el descenso gradiente lo optimice: al igual que PReLU, esto puede hacer que su modelo sea más potente, pero también corre el riesgo de sobreajustar los datos.

# Funciones de activación

- Entonces, ¿qué función de activación debes utilizar para las capas ocultas de tus redes neuronales profundas?
- ReLU sigue siendo una buena opción por defecto para tareas sencillas: a menudo es tan buena como las funciones de activación más sofisticadas, además es muy rápida de calcular y muchas bibliotecas y aceleradores de hardware proporcionan optimizaciones específicas para ReLU.
- Sin embargo, Swish es probablemente un mejor valor predeterminado para tareas más complejas, e incluso puedes probar Swish parametrizado con un parámetro  $\beta$  aprendible para las tareas más complejas.
- Mish puede dar resultados ligeramente mejores, pero requiere un poco más de cálculo. Si te preocupa mucho la latencia en tiempo de ejecución, quizá prefieras ReLU con fugas o ReLU con fugas parametrizado para tareas más complejas.
- Para MLPs profundos, prueba SELU, pero asegúrate de respetar las restricciones enumeradas anteriormente. Si dispones de tiempo y potencia de cálculo, puede utilizar la validación cruzada para evaluar también otras funciones de activación.

# Normalización por lotes (Batch Normalization)

- En un artículo de 2015, Sergey Ioffe y Christian Szegedy propusieron una técnica llamada normalización por lotes (BN) que aborda el peligro de que los problemas de gradientes que se desvanecen/explotan vuelvan durante el entrenamiento.
- La técnica consiste en añadir una operación en el modelo justo antes o después de la función de activación de cada capa oculta. Esta operación simplemente centra en cero y normaliza cada entrada, luego escala y desplaza el resultado utilizando dos nuevos vectores de parámetros por capa: uno para la escala y otro para el desplazamiento.
- En otras palabras, la operación permite al modelo aprender la escala y la media óptimas de cada una de las entradas de la capa.
- En muchos casos, si añades una capa BN como la primera capa de tu red neuronal, no necesitas estandarizar tu conjunto de entrenamiento. Es decir, no hay necesidad de StandardScaler o Normalization; la capa BN lo hará por ti (bueno, aproximadamente, ya que sólo mira un lote a la vez, y también puede reescalar y desplazar cada característica de entrada).



# Normalización por lotes

- Para centrar a cero y normalizar las entradas, el algoritmo necesita estimar la media y la desviación estándar de cada entrada. Para ello, evalúa la media y la desviación típica de la entrada en el minilote actual (de ahí el nombre de "normalización por lotes").

- Paso a paso en:

1. 
$$\boldsymbol{\mu}_B = \frac{1}{m_B} \sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

2. 
$$\sigma_B^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_B)^2$$

3. 
$$\hat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \boldsymbol{\mu}_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

4. 
$$\mathbf{z}^{(i)} = \boldsymbol{\gamma} \otimes \hat{\mathbf{x}}^{(i)} + \boldsymbol{\beta}$$

- $\boldsymbol{\mu}_B$  es el vector de medias de entrada, evaluado sobre todo el minilote  $B$  (contiene una media por entrada).
- $m_B$  es el número de instancias del minilote.
- $\boldsymbol{\sigma}_B$  es el vector de desviaciones estándar de entrada, también evaluado sobre todo el minilote (contiene una desviación estándar por entrada).
- $\hat{\mathbf{x}}^{(i)}$  es el vector de entradas centrado en cero y normalizado para la instancia  $i$ .
- $\varepsilon$  es un número minúsculo que evita la división por cero y garantiza que los gradientes no crezcan demasiado (normalmente  $10^{-5}$ ). Se denomina término de suavizado.
- $\boldsymbol{\gamma}$  es el vector de parámetros de escala de salida de la capa (contiene un parámetro de escala por entrada).
- $\otimes$  representa la multiplicación por elementos (cada entrada se multiplica por su correspondiente parámetro de escala de salida).
- $\boldsymbol{\beta}$  es el vector de parámetros de desplazamiento (offset) de salida de la capa (contiene un parámetro de desplazamiento por entrada). A cada entrada le corresponde un parámetro de desplazamiento.
- $\mathbf{z}^{(i)}$  es la salida de la operación BN. Es una versión reescalada y desplazada de las entradas.

# Normalización por lotes

- Así, durante el entrenamiento, la BN estandariza sus entradas y, a continuación, las reescala y las compensa.
- ¡Bien! ¿Y a la hora del test? Pues no es tan sencillo. De hecho, es posible que tengamos que hacer predicciones para instancias individuales en lugar de para lotes de instancias: en este caso, no tendremos forma de calcular la media y la desviación estándar de cada entrada.
- Además, aunque dispongamos de un lote de instancias, puede que sea demasiado pequeño, o que las instancias no sean independientes e idénticamente distribuidas, por lo que el cálculo de estadísticas sobre las instancias del lote no sería fiable.
- Una solución podría ser esperar hasta el final del entrenamiento y, a continuación, ejecutar todo el conjunto de entrenamiento a través de la red neuronal y calcular la media y la desviación estándar de cada entrada de la capa BN.
- Estas medias y desviaciones estándar de entrada "finales" podrían utilizarse en lugar de las medias y desviaciones estándar de entrada por lotes a la hora de hacer predicciones. Sin embargo, la mayoría de las implementaciones de normalización por lotes estiman estas estadísticas finales durante el entrenamiento utilizando una media móvil de las medias y desviaciones estándar de entrada de la capa.

# Normalización por lotes

- Esto es lo que Keras hace automáticamente cuando se utiliza la capa *BatchNormalization*.
- En resumen, en cada capa normalizada por lotes se aprenden cuatro vectores de parámetros:  $\gamma$  (el vector de escala de salida) y  $\beta$  (el vector de desplazamiento de salida) se aprenden mediante retropropagación normal, y  $\mu$  (el vector de media de entrada final) y  $\sigma$  (el vector de desviación estándar de entrada final) se estiman mediante una media móvil exponencial. Tenga en cuenta que  $\mu$  y  $\sigma$  se estiman durante el entrenamiento, pero sólo se utilizan después del entrenamiento (para sustituir las medias y desviaciones estándar de entrada por lotes en la ecuación anterior).
- Según los autores, *"aplicada a un modelo de clasificación de imágenes de última generación, la normalización por lotes logra la misma precisión con 14 veces menos pasos de entrenamiento y supera al modelo original por un margen significativo. [...] Utilizando un conjunto de redes normalizadas por lotes, mejoramos el mejor resultado publicado sobre clasificación de ImageNet: alcanzamos un error de validación del 4,9% sobre 5 (y un error de prueba del 4,8%), superando la precisión de los evaluadores humanos."*

# Normalización por lotes

- Por último, la normalización por lotes actúa como un regularizador, reduciendo la necesidad de otras técnicas de regularización (como el dropout).
- Sin embargo, la normalización por lotes añade cierta complejidad al modelo. Además, hay una penalización en el tiempo de ejecución: la red neuronal hace predicciones más lentas debido a los cálculos adicionales necesarios en cada capa.
- Afortunadamente, a menudo es posible fusionar la capa BN con la capa anterior después del entrenamiento, evitando así la penalización por tiempo de ejecución. Esto se hace actualizando los pesos y sesgos de la capa anterior para que produzca directamente salidas con la escala y el desplazamiento adecuados.
- Por ejemplo, si la capa anterior calcula  $\mathbf{XW} + \mathbf{b}$ , entonces la capa BN calculará  $\gamma \otimes (\mathbf{XW} + \mathbf{b} - \boldsymbol{\mu}) / \sigma + \boldsymbol{\beta}$  (ignorando el término de suavizado  $\epsilon$  en el denominador).
- Si definimos  $\mathbf{W}' = \gamma \otimes \mathbf{W} / \sigma$  y  $\mathbf{b}' = \gamma \otimes (\mathbf{b} - \boldsymbol{\mu}) / \sigma + \boldsymbol{\beta}$ , la ecuación se simplifica a  $\mathbf{XW}' + \mathbf{b}'$ . Por lo tanto, si sustituimos los pesos y sesgos de la capa anterior ( $\mathbf{W}$  y  $\mathbf{b}$ ) por los pesos y sesgos actualizados ( $\mathbf{W}'$  y  $\mathbf{b}'$ ), podemos deshacernos de la capa BN.

Código

# Normalización por lotes

- La clase *BatchNormalization* tiene bastantes hiperparámetros que puede ajustar. Por lo general, los valores predeterminados estarán bien, pero es posible que ocasionalmente necesite ajustar el *momentum*.
- Este hiperparámetro es utilizado por la capa *BatchNormalization* cuando actualiza las medias móviles exponenciales; dado un nuevo valor  $\mathbf{v}$  (es decir, un nuevo vector de medias de entrada o desviaciones estándar calculadas sobre el lote actual), la capa actualiza la media móvil  $\mathbf{v}$  utilizando la siguiente ecuación:

$$\hat{\mathbf{v}} \leftarrow \hat{\mathbf{v}} \times \text{momentum} + \mathbf{v} \times (1 - \text{momentum})$$

- Un buen valor de impulso suele ser cercano a 1; por ejemplo, 0,9, 0,99 o 0,999. Conviene tener más 9s para conjuntos de datos más grandes y para minilotes más pequeños.

# Normalización por lotes

- Otro hiperparámetro importante es el *eje*: determina qué eje debe normalizarse. Su valor por defecto es -1, lo que significa que, por defecto, se normalizará el último eje (utilizando las medias y desviaciones estándar calculadas en los otros ejes).
- Cuando el lote de entrada es 2D (es decir, la forma del lote es [*tamaño del lote, características*]), esto significa que cada característica de entrada se normalizará basándose en la media y la desviación estándar calculadas en todas las instancias del lote. Por ejemplo, la primera capa BN en el ejemplo MNIST normalizará independientemente (y reescalará y desplazará) cada una de las 784 características de entrada. Si movemos la primera capa BN antes de la capa Flatten, entonces los lotes de entrada serán 3D, con forma [*tamaño del lote, altura, anchura*]; por lo tanto, la capa BN calculará 28 medias y 28 desviaciones estándar (1 por columna de píxeles, calculada a través de todas las instancias en el lote y a través de todas las filas en la columna), y normalizará todos los píxeles en una columna dada usando la misma media y desviación estándar. También habrá sólo 28 parámetros de escala y 28 parámetros de desplazamiento. Si en lugar de ello desea tratar cada uno de los 784 píxeles de forma independiente, entonces debe establecer  $axis=[1, 2]$ .
- La normalización por lotes se ha convertido en una de las capas más utilizadas en las redes neuronales profundas, especialmente en las redes neuronales convolucionales profundas, hasta el punto de que a menudo se omite en los diagramas de arquitectura: se supone que la BN se añade después de cada capa.

# Recorte de degradado (Gradient Clipping)

- Otra técnica para mitigar el problema de la explosión de gradientes consiste en recortar los gradientes durante la retropropagación para que nunca superen cierto umbral. Esto se denomina *recorte de gradientes*.
- Esta técnica se utiliza generalmente en redes neuronales recurrentes, donde el uso de la normalización por lotes es complicado.

Código

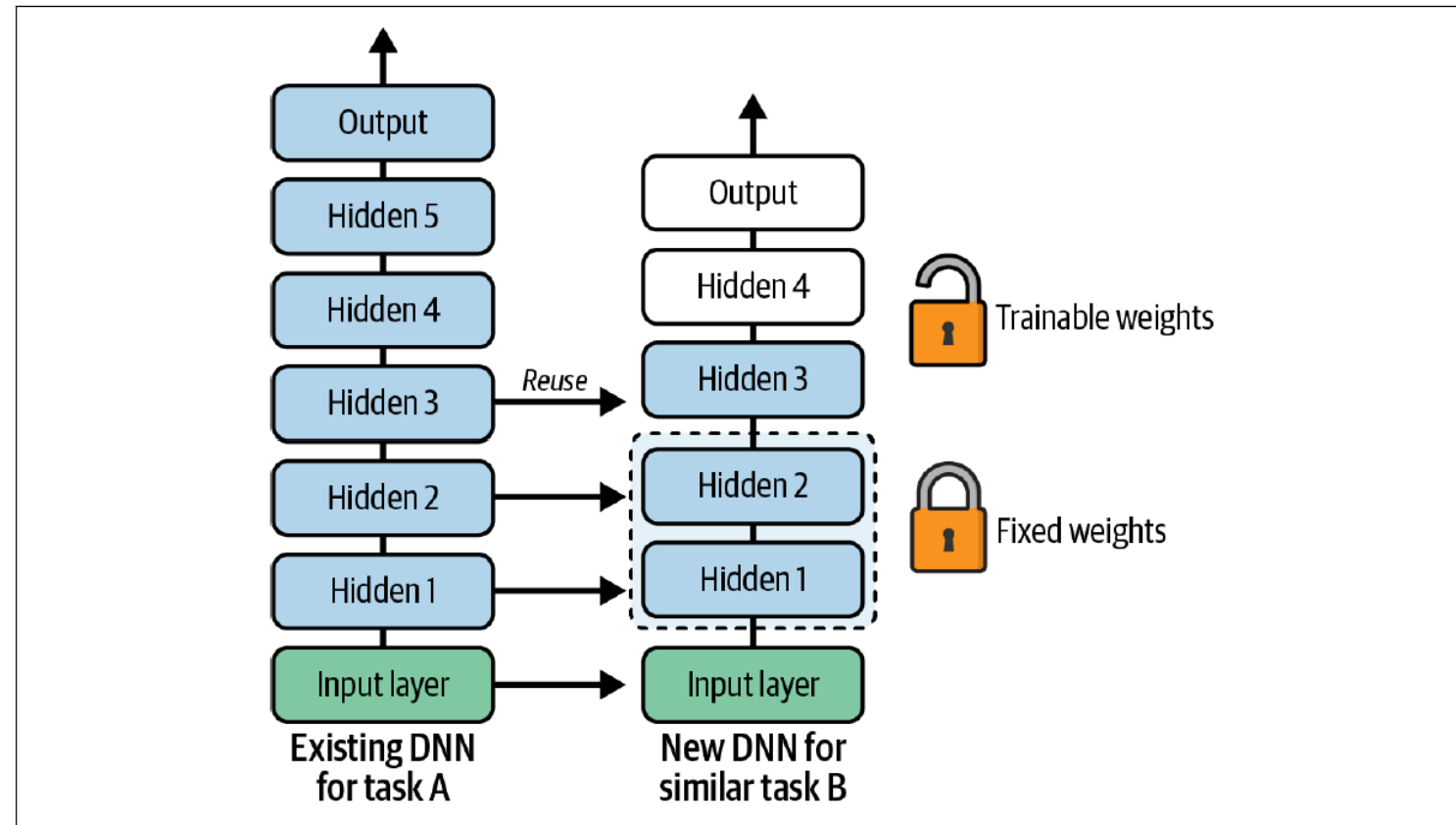


# Reutilización de capas preentrenadas

- Por lo general, no es una buena idea entrenar una DNN muy grande desde cero sin antes intentar encontrar una red neuronal existente que realice una tarea similar a la que se pretende abordar.
- Si encuentra una red neuronal de este tipo, generalmente podrá reutilizar la mayoría de sus capas, excepto las superiores. Esta técnica se llama *aprendizaje por transferencia*. No sólo acelera considerablemente el entrenamiento, sino que también requiere muchos menos datos de entrenamiento.
- Supongamos que tienes acceso a una DNN que fue entrenada para clasificar imágenes en 100 categorías diferentes, incluyendo animales, plantas, vehículos y objetos cotidianos, y ahora quieres entrenar una DNN para clasificar tipos específicos de vehículos. Estas tareas son muy similares, incluso se solapan parcialmente, por lo que deberías intentar reutilizar partes de la primera red.
- Si las imágenes de entrada para tu nueva tarea no tienen el mismo tamaño que las utilizadas en la tarea original, normalmente tendrá que añadir un paso de preprocesamiento para redimensionarlas al tamaño esperado por el modelo original. En general, el aprendizaje por transferencia funciona mejor cuando las entradas tienen características de bajo nivel similares.

# Reutilización de capas preentrenadas

- Por lo general, la capa de salida del modelo original debe sustituirse porque lo más probable es que no sea útil en absoluto para la nueva tarea, y probablemente no tendrá el número adecuado de salidas.



# Reutilización de capas preentrenadas

- Del mismo modo, es menos probable que las capas ocultas superiores del modelo original sean tan útiles como las capas inferiores, ya que las características de alto nivel más útiles para la nueva tarea pueden diferir significativamente de las que eran más útiles para la tarea original. Se trata de encontrar el número adecuado de capas para reutilizar.
- Prueba primero a congelar todas las capas reutilizadas (es decir, haz que sus pesos no sean entrenables para que el descenso gradiente no los modifique y permanezcan fijos), luego entrena tu modelo y comprueba su rendimiento. A continuación, prueba a descongelar una o dos de las capas ocultas superiores para que la retropropagación las modifique y comprueba si mejora el rendimiento. Cuantos más datos de entrenamiento tengas, más capas podrás descongelar. También es útil reducir la tasa de aprendizaje cuando descongele capas reutilizadas: esto evitará que se estropeen sus pesos ajustados.
- Si sigues sin obtener un buen rendimiento y dispones de pocos datos de entrenamiento, intenta eliminar la capa o capas ocultas superiores y vuelve a congelar todas las capas ocultas restantes.
- Puedes iterar hasta que encuentre el número adecuado de capas para reutilizar. Si tienes muchos datos de entrenamiento, puedes intentar sustituir las capas ocultas superiores en lugar de eliminarlas, e incluso añadir más capas ocultas.

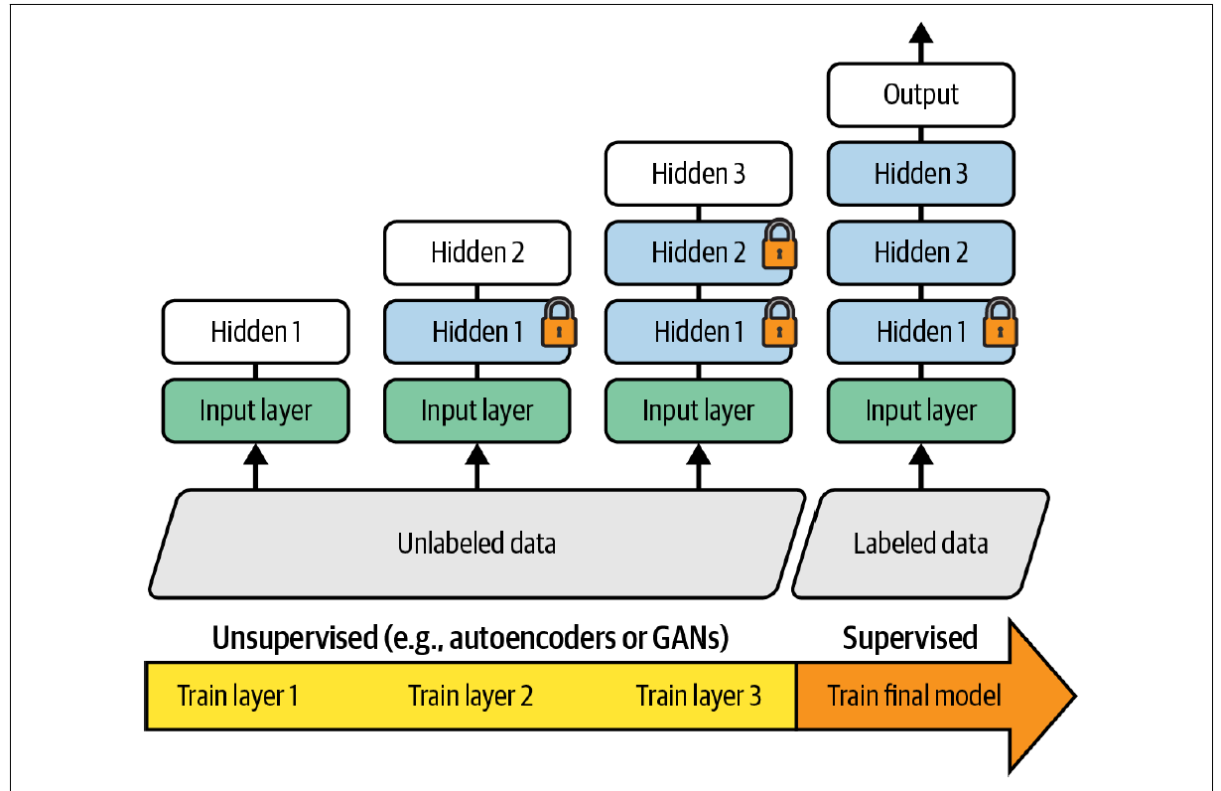
Código

# PreTraining no supervisado

- Supongamos que deseas abordar una tarea compleja para la que no dispones de muchos datos de entrenamiento etiquetados, pero lamentablemente no puedes encontrar un modelo entrenado en una tarea similar.
- ¡No pierdas la esperanza! En primer lugar, deberías intentar recopilar más datos de entrenamiento etiquetados, pero si no se puede, aún puedes realizar un preentrenamiento no supervisado (véase la siguiente figura).
- De hecho, a menudo es barato reunir ejemplos de entrenamiento sin etiquetar, pero es caro etiquetarlos. Si puedes reunir muchos datos de entrenamiento sin etiquetar, puedes intentar utilizarlos para entrenar un modelo no supervisado, como un autoencoder o una red generativa adversaria (GAN).
- A continuación, puedes reutilizar las capas inferiores del autoencoder o las capas inferiores del discriminador de la GAN, añadir la capa de salida para su tarea en la parte superior y ajustar la red final utilizando el aprendizaje supervisado (es decir, con los ejemplos de entrenamiento etiquetados).

# PreTraining no supervisado

- Ten en cuenta que en los primeros días del aprendizaje profundo era difícil entrenar modelos profundos, por lo que la gente utilizaba una técnica llamada preentrenamiento greedy por capas (esta figura). Primero se entrenaba un modelo no supervisado con una sola capa, normalmente un RBM, luego se congelaba esa capa y se añadía otra encima, se volvía a entrenar el modelo (de hecho, solo se entrenaba la nueva capa), luego se congelaba la nueva capa y se añadía otra encima, se volvía a entrenar el modelo, y así sucesivamente.
- Hoy en día, las cosas son mucho más sencillas: la gente suele entrenar todo el modelo no supervisado de una sola vez y utiliza autoencoders o GAN en lugar de RBM.



# PreTraining no supervisado

- Es esta técnica la que Geoffrey Hinton y su equipo utilizaron en 2006, y que propició el renacimiento de las redes neuronales y el éxito del aprendizaje profundo.
- Hasta 2010, el preentrenamiento no supervisado -típicamente con máquinas de Boltzmann restringidas (RBM)- era la norma para las redes profundas, y sólo después de que se aliviara el problema de los gradientes de fuga se hizo mucho más común entrenar las DNN utilizando puramente el aprendizaje supervisado.
- El preentrenamiento no supervisado (que hoy en día suele utilizar autoencoders o GANs en lugar de RBMs) sigue siendo una buena opción cuando se tiene una tarea compleja que resolver, ningún modelo similar que se pueda reutilizar y pocos datos de entrenamiento etiquetados pero muchos datos de entrenamiento sin etiquetar.

# PreTraining en una tarea auxiliar

- Si no dispones de muchos datos de entrenamiento etiquetados, una última opción es entrenar una primera red neuronal en una tarea auxiliar para la que pueda obtener o generar fácilmente datos de entrenamiento etiquetados y, a continuación, reutilizar las capas inferiores de esa red para su tarea real. Las capas inferiores de la primera red neuronal aprenderán detectores de características que probablemente serán reutilizables por la segunda red neuronal.
- Por ejemplo, si se quiere crear un sistema para reconocer caras, es posible que sólo se disponga de unas pocas fotos de cada persona, lo que resulta claramente insuficiente para entrenar un buen clasificador. Reunir cientos de fotos de cada persona no sería práctico. Sin embargo, se podrían recopilar muchas fotos de personas al azar en Internet y entrenar una primera red neuronal para detectar si dos fotos diferentes muestran o no a la misma persona. Una red de este tipo aprendería buenos detectores de rostros, por lo que reutilizar sus capas inferiores le permitiría entrenar un buen clasificador de rostros con pocos datos de entrenamiento.



# PreTraining en una tarea auxiliar

- Para aplicaciones de procesamiento del lenguaje natural (PLN), puedes descargar un corpus de millones de documentos de texto y generar automáticamente datos etiquetados a partir de él. Por ejemplo, puede enmascarar aleatoriamente algunas palabras y entrenar un modelo para que prediga cuáles son las palabras que faltan (por ejemplo, debería predecir que la palabra que falta en la frase "What \_\_\_ you saying?" es probablemente "are" o "were"). Si puede entrenar un modelo para que obtenga un buen rendimiento en esta tarea, entonces ya sabrá bastante sobre el lenguaje, y sin duda podrá reutilizarlo para su tarea real y perfeccionarlo con sus datos etiquetados.
- *El aprendizaje autosupervisado* consiste en generar automáticamente las etiquetas a partir de los propios datos, como en el ejemplo del enmascaramiento de texto, y después entrenar un modelo en el conjunto de datos "etiquetados" resultante mediante técnicas de aprendizaje supervisado.

# Optimizadores Más Rápidos

- Entrenar una red neuronal profunda muy grande puede ser dolorosamente lento.
- Hasta ahora hemos visto cuatro formas de acelerar el entrenamiento (y llegar a una solución mejor):
  - aplicar una buena estrategia de inicialización de los pesos de conexión
  - utilizando una buena función de activación
  - utilizando la normalización por lotes
  - reutilizar partes de una red preentrenada (posiblemente construida para una tarea auxiliar o mediante aprendizaje no supervisado).
- Otro enorme aumento de la velocidad procede de la utilización de un optimizador más rápido que el optimizador de descenso de gradiente normal. Presentaremos los algoritmos de optimización más populares: momentum, gradiente acelerado de Nesterov, AdaGrad, RMSProp y, por último, Adam y sus variantes.

# Optimizadores Más Rápidos: Momentum

- Imaginemos una bola de bolos rodando por una pendiente suave sobre una superficie lisa: empezará despacio, pero cogerá impulso rápidamente hasta que alcance la velocidad terminal (si hay algo de fricción o resistencia del aire). Esta es la idea central de la optimización del momentum, propuesta por Boris Polyak en 1964.
- Por el contrario, el descenso por gradiente regular dará pequeños pasos cuando la pendiente sea suave y grandes pasos cuando la pendiente sea pronunciada, pero nunca cogerá velocidad. Como resultado, el descenso de gradiente regular es generalmente mucho más lento para alcanzar el mínimo que la optimización del momentum.
- Recordemos que el descenso de gradiente actualiza los pesos  $\theta$  restando directamente el gradiente de la función de coste  $J(\theta)$  con respecto a los pesos ( $\nabla_{\theta} J(\theta)$ ) multiplicado por la tasa de aprendizaje  $\eta$ .
- La ecuación es  $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$ . No le importa cuáles eran los gradientes anteriores. Si el gradiente local es pequeño, va muy despacio.

# Optimizadores Más Rápidos: Momentum

- La optimización del momentum se preocupa mucho de cuáles eran los gradientes anteriores: en cada iteración, resta el gradiente local del vector *momentum*  $\mathbf{m}$  (multiplicado por la tasa de aprendizaje  $\eta$ ), y actualiza los pesos añadiendo este vector momentum (véase la siguiente ecuación).
- En otras palabras, el gradiente se utiliza como aceleración, no como velocidad. Para simular algún tipo de mecanismo de fricción y evitar que el impulso crezca demasiado, el algoritmo introduce un nuevo hiperparámetro  $\beta$ , denominado *momentum*, que debe fijarse entre 0 (alta fricción) y 1 (sin fricción). Un valor típico de momentum es 0.9.

$$1. \quad \mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

$$2. \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{m}$$

# Optimizadores Más Rápidos: Momentum

- Se puede comprobar que si el gradiente permanece constante, la velocidad terminal (es decir, el tamaño máximo de las actualizaciones de peso) es igual a ese gradiente multiplicado por la tasa de aprendizaje  $\eta$  multiplicado por  $1 / (1 - \beta)$  (ignorando el signo).
- Por ejemplo, si  $\beta = 0.9$ , entonces la velocidad terminal es igual a 10 veces el gradiente por la tasa de aprendizaje, ¡así que la optimización por momentum acaba yendo 10 veces más rápido que el descenso por gradiente! Esto permite a la optimización por momentum escapar de las mesetas mucho más rápido que el descenso por gradiente.
- Debido al momentum, el optimizador puede pasarse un poco, luego volver, pasarse otra vez y oscilar así muchas veces antes de estabilizarse en el mínimo. Esta es una de las razones por lo que es bueno tener un poco de fricción en el sistema: se deshace de estas oscilaciones y por lo tanto acelera la convergencia.
- El único inconveniente de la optimización del momentum es que añade otro hiperparámetro más que hay que ajustar. Sin embargo, el valor de momentum de 0.9 suele funcionar bien en la práctica y casi siempre va más rápido que el descenso por gradiente normal.

# Gradiente acelerado Nesterov

- Una pequeña variante de la optimización del momentum, propuesta por Yurii Nesterov en 1983, es casi siempre más rápida que la optimización del momentum normal. El método del *gradiente acelerado de Nesterov* (NAG), también conocido como *optimización del momentum de Nesterov*, mide el gradiente de la función de coste no en la posición local  $\theta$ , sino ligeramente por delante en la dirección del momento, en  $\theta + \beta \mathbf{m}$ .

$$1. \quad \mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta + \beta \mathbf{m})$$

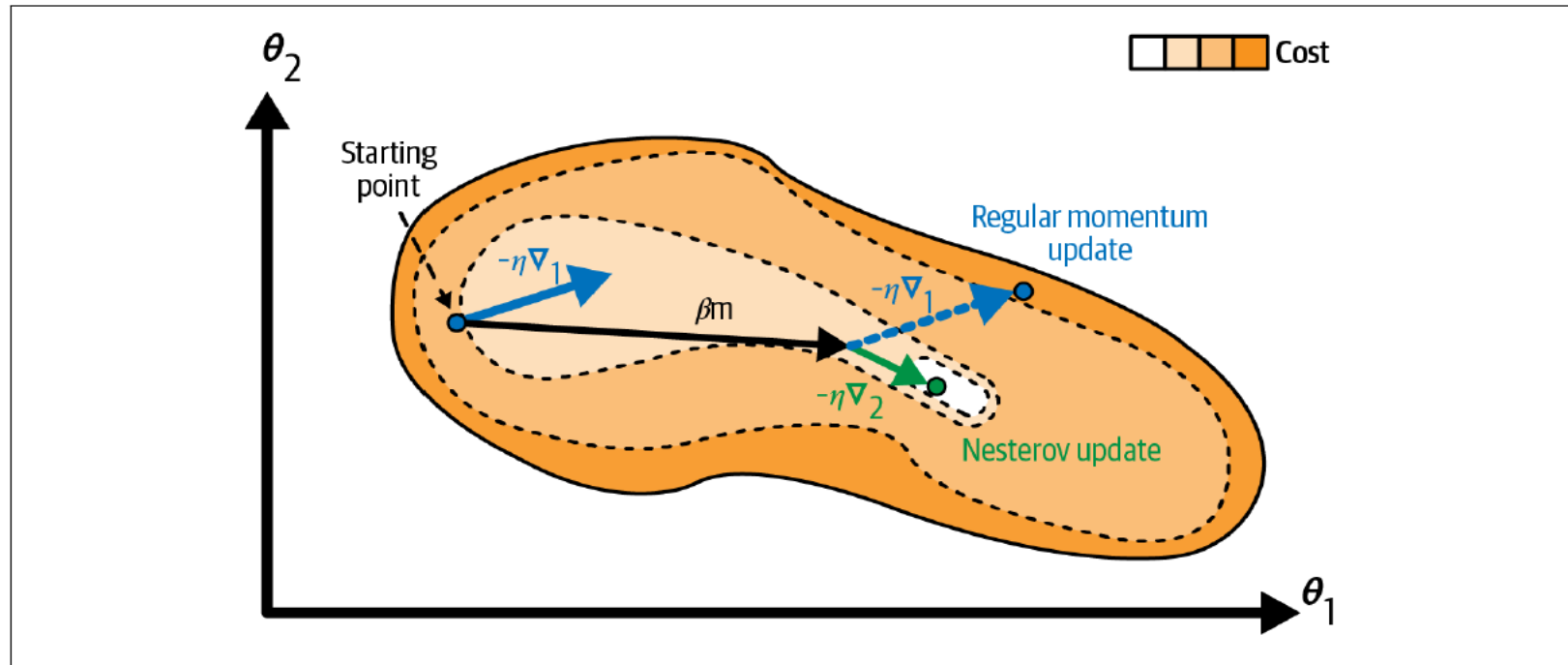
$$2. \quad \theta \leftarrow \theta + \mathbf{m}$$

- Este pequeño ajuste funciona porque, en general, el vector momento apuntará en la dirección correcta (es decir, hacia el óptimo), por lo que será ligeramente más preciso utilizar el gradiente medido un poco más lejos en esa dirección que el gradiente en la posición original.

Yurii Nesterov, "A Method for Unconstrained Convex Minimization Problem with the Rate of Convergence  $O(1/k^2)$ ,"  
Doklady AN USSR 269 (1983): 543-547.

# Gradiente acelerado Nesterov

- Como puedes ver aquí (donde  $\nabla_1$  representa el gradiente de la función de coste medido en el punto inicial  $\theta$ , y  $\nabla_2$  representa el gradiente en el punto situado en  $\theta + \beta m$ ).





# AdaGrad

- Consideremos de nuevo el problema del cuenco alargado: el descenso por gradiente comienza bajando rápidamente por la pendiente más pronunciada, que no apunta directamente hacia el óptimo global, y luego desciende muy lentamente hasta el fondo del valle.
- Sería bueno que el algoritmo pudiera corregir su dirección antes para apuntar un poco más hacia el óptimo global. El algoritmo AdaGrad consigue esta corrección reduciendo la escala del vector gradiente a lo largo de las cotas más empinadas.

$$1. \quad \mathbf{s} \leftarrow \mathbf{s} + \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

$$2. \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \oslash \sqrt{\mathbf{s} + \boldsymbol{\varepsilon}}$$

- El primer paso acumula el cuadrado de los gradientes en el vector  $\mathbf{s}$  (recuérdese que el símbolo  $\otimes$  representa la multiplicación por elementos). Esta forma vectorizada equivale a calcular  $s_i \leftarrow s_i + (\partial J(\boldsymbol{\theta}) / \partial \theta_i)^2$  para cada elemento  $s_i$  del vector  $\mathbf{s}$ ; en otras palabras, cada  $s_i$  acumula los cuadrados de la derivada parcial de la función de coste con respecto al parámetro  $\theta_i$ . Si la función de coste es empinada a lo largo de la dimensión  $i^{\text{th}}$ , entonces  $s_i$  será cada vez mayor en cada iteración.

# AdaGrad

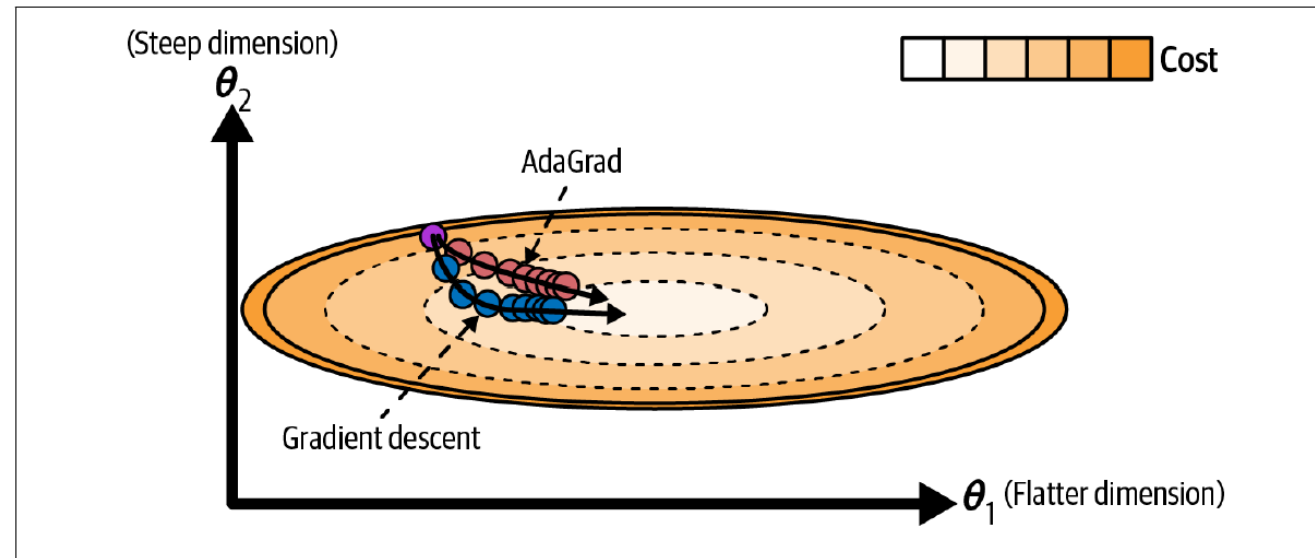
- El segundo paso es casi idéntico al descenso por gradiente, pero con una gran diferencia: el vector del gradiente se reduce en un factor de  $\frac{1}{\sqrt{s_i + \epsilon}}$  (el símbolo  $\oslash$  representa la división por elementos, y  $\epsilon$  es un término de suavizado para evitar la división por cero, normalmente fijado en  $10^{-10}$ ). Esta forma vectorizada equivale a calcular simultáneamente para todos los parámetros  $\theta_i$  esta expresión:

$$\theta_i \leftarrow \theta_i - \eta \partial J(\boldsymbol{\theta}) / \partial \theta_i / \sqrt{s_i + \epsilon}$$

- En resumen, este algoritmo hace decaer la tasa de aprendizaje, pero lo hace más rápido para dimensiones empinadas que para dimensiones con pendientes más suaves. Esto se denomina tasa de aprendizaje adaptativa. Ayuda a dirigir las actualizaciones resultantes más directamente hacia el óptimo global (véase la siguiente figura). Otra ventaja es que requiere ajustar mucho menos el hiperparámetro  $\eta$  de la tasa de aprendizaje.

# AdaGrad

- AdaGrad suele funcionar bien para problemas cuadráticos sencillos, pero a menudo se detiene demasiado pronto al entrenar redes neuronales: la tasa de aprendizaje se reduce tanto que el algoritmo acaba deteniéndose por completo antes de alcanzar el óptimo global. Así que aunque Keras tiene un optimizador Adagrad, no deberías usarlo para entrenar redes neuronales profundas (aunque puede ser eficiente para tareas más simples como la regresión lineal). Aún así, entender AdaGrad es útil para comprender los otros optimizadores de tasa de aprendizaje adaptativo.



# RMSProp

- Como hemos visto, AdaGrad corre el riesgo de ralentizarse demasiado rápido y no converger nunca al óptimo global. El algoritmo RMSProp soluciona esto acumulando sólo los gradientes de las iteraciones más recientes, en lugar de todos los gradientes desde el inicio del entrenamiento. Para ello, utiliza un decaimiento exponencial en el primer paso.

$$1. \quad \mathbf{s} \leftarrow \rho \mathbf{s} + (1 - \rho) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

$$2. \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \oslash \sqrt{\mathbf{s} + \varepsilon}$$

- La tasa de decaimiento  $\rho$  suele fijarse en 0.9. Sí, se trata una vez más de un nuevo hiperparámetro, pero este valor por defecto suele funcionar bien, por lo que puede que no sea necesario ajustarlo en absoluto.
- Excepto en problemas muy simples, este optimizador casi siempre funciona mucho mejor que AdaGrad. De hecho, era el algoritmo de optimización preferido de muchos investigadores hasta que apareció la optimización Adam.

# Adam

- *Adam*, que significa *estimación adaptativa de momentos*, combina las ideas de la optimización de momentos y RMSProp: al igual que la optimización de momentos, realiza un seguimiento de una media exponencialmente decreciente de gradientes pasados; y al igual que RMSProp, realiza un seguimiento de una media exponencialmente decreciente de gradientes pasados al cuadrado.
- Se trata de estimaciones de la media y la varianza (no centrada) de los gradientes. La media suele denominarse primer momento, mientras que la varianza suele denominarse segundo momento, de ahí el nombre del algoritmo.

# Adam

1.  $\mathbf{m} \leftarrow \beta_1 \mathbf{m} - (1 - \beta_1) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
2.  $\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
3.  $\widehat{\mathbf{m}} \leftarrow \frac{\mathbf{m}}{1 - \beta_1^t}$
4.  $\widehat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \beta_2^t}$
5.  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \widehat{\mathbf{m}} \oslash \sqrt{\widehat{\mathbf{s}} + \varepsilon}$

- En esta ecuación,  $t$  representa el número de iteración (empezando por 1).
- Si nos fijamos en los pasos 1, 2 y 5, nos daremos cuenta de la gran similitud de Adam tanto con la optimización del momento como con RMSProp:  $\beta_1$  corresponde a  $\beta$  en la optimización del momento, y  $\beta_2$  corresponde a  $\rho$  en RMSProp. La única diferencia es que el paso 1 calcula una media exponencialmente decreciente en lugar de una suma exponencialmente decreciente, pero en realidad son equivalentes salvo por un factor constante (la media decreciente es sólo  $1 - \beta_1$  veces la suma decreciente).
- Los pasos 3 y 4 son un detalle técnico: dado que  $\mathbf{m}$  y  $\mathbf{s}$  se inicializan en 0, estarán sesgados hacia 0 al principio del entrenamiento, por lo que estos dos pasos ayudarán a aumentar  $\mathbf{m}$  y  $\mathbf{s}$  al principio del entrenamiento.

# Adam

- El hiperparámetro de decaimiento del momento  $\beta_1$  suele inicializarse en 0.9, mientras que el hiperparámetro de decaimiento de escala  $\beta_2$  suele inicializarse en 0.999. Como en el caso anterior, el término de suavizado  $\varepsilon$  suele inicializarse con un número pequeño, como  $10^{-7}$ .
- Dado que Adam es un algoritmo de tasa de aprendizaje adaptativo, como AdaGrad y RMSProp, requiere menos ajuste del hiperparámetro de tasa de aprendizaje  $\eta$ . A menudo se puede utilizar el valor por defecto  $\eta = 0.001$ , por lo que Adam aún más fácil de usar que el descenso por gradiente.
- Merece la pena mencionar tres variantes de Adam: AdaMax, Nadam y AdamW.



# AdaMax

- AdaMax sustituye la norma  $\ell_2$  por la norma  $\ell_\infty$  (una forma elegante de decir el máximo). En  $\hat{\mathbf{s}} \leftarrow \max(\beta_2 \hat{\mathbf{s}}, \text{abs}(\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})))$ , se sustituye el paso 2 en la ecuación de Adam con  $\hat{\mathbf{s}} \leftarrow \max(\beta_2 \hat{\mathbf{s}}, \text{abs}(\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})))$ , abandona el paso 4, y en el paso 5 reduce las actualizaciones del gradiente en un factor de  $\hat{\mathbf{s}}$ , que es el máximo del valor absoluto de los gradientes retardados.
- En la práctica, esto puede hacer que AdaMax sea más estable que Adam, pero realmente depende del conjunto de datos, y en general Adam rinde mejor. Por lo tanto, este es sólo un optimizador más que puede probar si experimenta problemas con Adam en alguna tarea.

# Nadam y AdamW

- **Nadam:** La optimización Nadam es la optimización Adam más el truco Nesterov, por lo que a menudo convergerá ligeramente más rápido que Adam.
- **AdamW:** es una variante de Adam que integra una técnica de regularización llamada decaimiento del peso. El decaimiento del peso reduce el tamaño de los pesos del modelo en cada iteración de entrenamiento multiplicándolos por un factor de decaimiento como 0,99.
- Esto puede recordarte a la regularización  $\ell_2$ , que también tiene como objetivo mantener los pesos pequeños, y de hecho se puede demostrar matemáticamente que la regularización  $\ell_2$  es equivalente al decaimiento del peso cuando se utiliza SGD.
- Sin embargo, cuando se utiliza Adam o sus variantes, la regularización  $\ell_2$  y el decaimiento del peso no son equivalentes: en la práctica, la combinación de Adam con la regularización  $\ell_2$  da lugar a modelos que a menudo no generalizan tan bien como los producidos por SGD.
- AdamW soluciona este problema combinando adecuadamente a Adam con el decaimiento del peso.

# Acerca de la optimización compleja

- Todas las técnicas de optimización analizadas hasta ahora sólo se basan en las *derivadas parciales de primer orden (jacobianos)*.
- La literatura sobre optimización también contiene asombrosos algoritmos basados en las *derivadas parciales de segundo orden* (los *Hessianos*, que son las derivadas parciales de las Jacobianas).
- Por desgracia, estos algoritmos son muy difíciles de aplicar a redes neuronales profundas porque hay  $n^2$  Hessianos por salida (donde  $n$  es el número de parámetros), en lugar de sólo  $n$  Jacobianos por salida.
- Como las DNN suelen tener decenas de miles de parámetros o más, los algoritmos de optimización de segundo orden a menudo ni siquiera caben en la memoria, e incluso cuando lo hacen, calcular los hessianos es demasiado lento.

# Entrenamiento de modelos dispersos

- Todos los algoritmos de optimización que acabamos de analizar producen modelos densos, lo que significa que la mayoría de los parámetros serán distintos de cero. Si necesitas un modelo rapidísimo en tiempo de ejecución, o si necesitas que ocupe menos memoria, puede que prefieras acabar con un modelo disperso.
- Una forma de conseguirlo es entrenar el modelo como de costumbre y, a continuación, deshacerse de los pesos diminutos (ponerlos a cero). Sin embargo, esto no suele conducir a un modelo muy disperso, y puede degradar el rendimiento del modelo.
- Una mejor opción es aplicar una fuerte regularización  $\ell_1$  durante el entrenamiento, ya que empuja al optimizador a reducir a cero tantos pesos como pueda (como se discute en "Regresión Lasso").
- Si estas técnicas siguen siendo insuficientes, echa un vistazo a TensorFlow Model Optimization Toolkit (TF-MOT), que proporciona una API de poda capaz de eliminar iterativamente conexiones durante el entrenamiento en función de su magnitud.

# Comparación de optimizadores

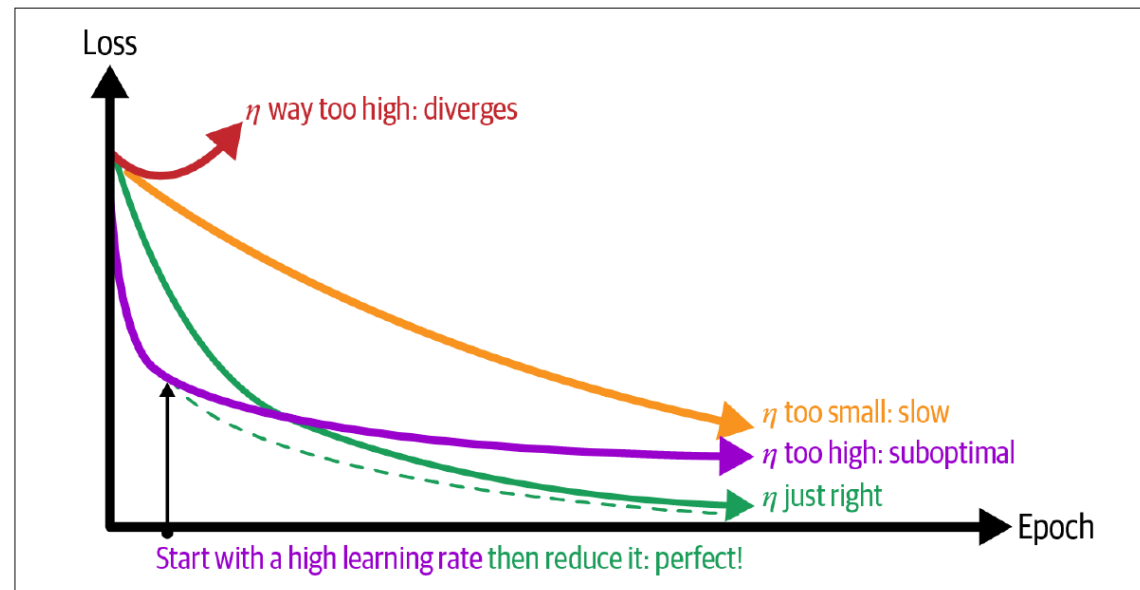
► (\* es malo, \*\* es mediocre y \*\*\* es bueno)

| Class                            | Convergence speed | Convergence quality |
|----------------------------------|-------------------|---------------------|
| SGD                              | *                 | ***                 |
| SGD(momentum=...)                | **                | ***                 |
| SGD(momentum=..., nesterov=True) | **                | ***                 |
| Adagrad                          | ***               | * (stops too early) |
| RMSprop                          | ***               | ** or ***           |
| Adam                             | ***               | ** or ***           |
| AdaMax                           | ***               | ** or ***           |
| Nadam                            | ***               | ** or ***           |
| AdamW                            | ***               | ** or ***           |

Código

# Programación del ritmo de aprendizaje

- Encontrar una buena tasa de aprendizaje es muy importante. Si la fijas demasiado alta, el entrenamiento puede divergir. Si la fijas demasiado baja, el entrenamiento acabará convergiendo al óptimo, pero tardará mucho tiempo. Si lo fijas ligeramente alto, al principio progresará muy rápido, pero acabará bailando alrededor del óptimo y nunca llegará a estabilizarse. Si dispone de una potencia de cálculo limitada, puede que tenga que interrumpir el entrenamiento antes de que haya convergido correctamente, lo que dará lugar a una solución subóptima.



# Programación del ritmo de aprendizaje

- Puedes encontrar una buena tasa de aprendizaje entrenando el modelo durante unos cientos de iteraciones, aumentando exponencialmente la tasa de aprendizaje desde un valor muy pequeño hasta un valor muy grande y, a continuación, observando la curva de aprendizaje y eligiendo una tasa de aprendizaje ligeramente inferior a aquella en la que la curva de aprendizaje comienza a dispararse de nuevo hacia arriba. Entonces puedes reinicializar tu modelo y entrenarlo con esa tasa de aprendizaje.
- Pero se puede hacer mejor que con una tasa de aprendizaje constante: si se empieza con una tasa de aprendizaje grande y luego se reduce una vez que el entrenamiento deja de progresar rápidamente, se puede llegar a una buena solución más rápido que con la tasa de aprendizaje constante óptima.
- Existen muchas estrategias diferentes para reducir la tasa de aprendizaje durante el entrenamiento. También puede ser beneficioso empezar con un ritmo de aprendizaje bajo, aumentarlo y luego volver a reducirlo. Estas estrategias se denominan programas de aprendizaje.



# Programación del ritmo de aprendizaje

## ■ Programación por energía (Power Scheduling)

- Establece la tasa de aprendizaje en función del número de iteración  $t$ :  $\eta(t) = \eta_0 / (1 + t/s)^c$ . La tasa de aprendizaje inicial  $\eta_0$ , la potencia  $c$  (normalmente fijada en 1) y el paso decay  $s$  son hiperparámetros. La tasa de aprendizaje disminuye en cada paso. Después de  $s$  pasos, la tasa de aprendizaje baja a  $\eta_0 / 2$ . Después de  $s$  pasos más baja a  $\eta_0 / 3$ , luego baja a  $\eta_0 / 4$ , luego a  $\eta_0 / 5$ , y así sucesivamente. Como se puede ver, este schedule primero cae rápidamente, entonces más y más lentamente. Por supuesto, la programación por energía requiere afinar  $\eta_0$  y  $s$  (y posiblemente  $c$ ).

## ■ Programación exponencial

- Establece la tasa de aprendizaje en  $\eta(t) = \eta_0 0.1^{t/s}$ . La tasa de aprendizaje se reducirá gradualmente en un factor de 10 cada  $s$  pasos. Mientras que la programación por energía reduce la tasa de aprendizaje cada vez más lentamente, la programación exponencial sigue reduciéndola en un factor de 10 cada  $s$  pasos.

# Programación del ritmo de aprendizaje

## ► Programación constante a trozos

- Utiliza una tasa de aprendizaje constante para un número de épocas (por ejemplo,  $\eta_0 = 0.1$  para 5 épocas), luego una tasa de aprendizaje menor para otro número de épocas (por ejemplo,  $\eta_1 = 0.001$  para 50 épocas), y así sucesivamente. Aunque esta solución puede funcionar muy bien, requiere jugar para averiguar la secuencia correcta de tasas de aprendizaje y cuánto tiempo utilizar cada una de ellas.

## ► Programación del rendimiento (Performance Scheduling)

- Mide el error de validación cada  $N$  pasos (igual que para la parada anticipada) y reduce la tasa de aprendizaje en un factor  $\lambda$  cuando el error deje de caer.

# Programación del ritmo de aprendizaje

## ► Programación de 1 ciclo

- Al contrario que los otros enfoques, comienza aumentando la tasa de aprendizaje inicial  $\eta_0$ , creciendo linealmente hasta  $\eta_1$  a mitad del entrenamiento. Después disminuye la tasa de aprendizaje linealmente hasta  $\eta_0$  de nuevo durante la segunda mitad del entrenamiento, terminando las últimas épocas bajando la tasa varios órdenes de magnitud (todavía linealmente). La tasa de aprendizaje máxima  $\eta_1$  se elige utilizando el mismo enfoque que usamos para encontrar la tasa de aprendizaje óptima, y la tasa de aprendizaje inicial  $\eta_0$  suele ser 10 veces menor. Cuando usamos un momentum, empezamos primero con un momentum alto (por ejemplo, 0,95), luego lo bajamos a un momentum más bajo durante la primera mitad del entrenamiento (por ejemplo, hasta 0,85, linealmente), y luego lo volvemos a subir al valor máximo (por ejemplo, 0,95) durante la segunda mitad del entrenamiento, terminando las últimas épocas con ese valor máximo.
- Smith realizó muchos experimentos que demostraron que este enfoque a menudo era capaz de acelerar considerablemente el entrenamiento y alcanzar un mejor rendimiento. Por ejemplo, en el popular conjunto de datos de imágenes CIFAR10, este enfoque alcanzó una precisión de validación del 91,9% en sólo 100 épocas, frente a una precisión del 90,3% en 800 épocas mediante un enfoque estándar (con la misma arquitectura de red neuronal). Esta hazaña se denominó *superconvergencia*.

# Programación del ritmo de aprendizaje

- Un artículo de 2013 de Andrew Senior et al. comparó el rendimiento de algunos de los programas de aprendizaje más populares cuando se utiliza la optimización del momento para entrenar redes neuronales profundas para el reconocimiento del habla.
- Los autores concluyeron que, en esta situación, tanto la programación de rendimiento como la programación exponencial funcionaban bien.
- Se decantaron por la programación exponencial porque era fácil de ajustar y convergía ligeramente más rápido a la solución óptima.
- También mencionaron que era más fácil de implementar que la programación del rendimiento, pero en Keras ambas opciones son fáciles.
- Dicho esto, el enfoque de 1 ciclo parece funcionar incluso mejor.

Código

# Evitar el sobreajuste mediante la regularización

- Ya hemos aplicado anteriormente una de las mejores técnicas de regularización: la detención anticipada (early-stopping).
- Además, aunque la normalización por lotes se diseñó para resolver los problemas de gradientes inestables, también actúa como un regularizador bastante bueno.
- En esta sección examinaremos otras técnicas populares de regularización para redes neuronales:  $\ell_1$  y  $\ell_2$  regularización, drop-out y regularización max-norm.
- Al igual que en los modelos lineales simples, puede utilizar la regularización  $\ell_2$  para restringir los pesos de conexión de una red neuronal, y/o la regularización  $\ell_1$  si desea un modelo disperso (con muchos pesos iguales a 0).

Código

# Abandono (Drop-out)

- *Dropout* es una de las técnicas de regularización más populares para redes neuronales profundas. Se propuso en un artículo de Geoffrey Hinton et al. en 2012 y se detalló en un artículo de Nitish Srivastava et al. de 2014, y ha demostrado ser muy eficaz: muchas redes neuronales de última generación utilizan dropout, ya que les proporciona un aumento de precisión del 1%-2%. Puede que no parezca mucho, pero cuando un modelo ya tiene una precisión del 95 %, conseguir un aumento de precisión del 2 % significa reducir la tasa de error en casi un 40 % (pasar de un 5 % de error a aproximadamente un 3 %).
- Es un algoritmo bastante sencillo: en cada paso de entrenamiento, cada neurona (incluidas las neuronas de entrada, pero excluyendo siempre las de salida) tiene una probabilidad  $p$  de ser "abandonada" temporalmente, lo que significa que será ignorada por completo durante este paso de entrenamiento, pero puede estar activa durante el siguiente. El hiperparámetro  $p$  se denomina tasa de abandono y se suele fijar entre el 10% y el 50%: entre el 20% y el 30% en redes neuronales recurrentes y entre el 40% y el 50% en redes neuronales convolucionales. Después del entrenamiento, las neuronas ya no se abandonan. Y eso es todo (excepto por un detalle técnico que discutiremos momentáneamente).

Geoffrey E. Hinton y otros, "Improving Neural Networks by Preventing Co-Adaptation of Feature Detectors", arXiv preprint arXiv:1207.0580 (2012).

Nitish Srivastava y otros, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", Journal of Machine Learning Research 15 (2014): 1929-1958.

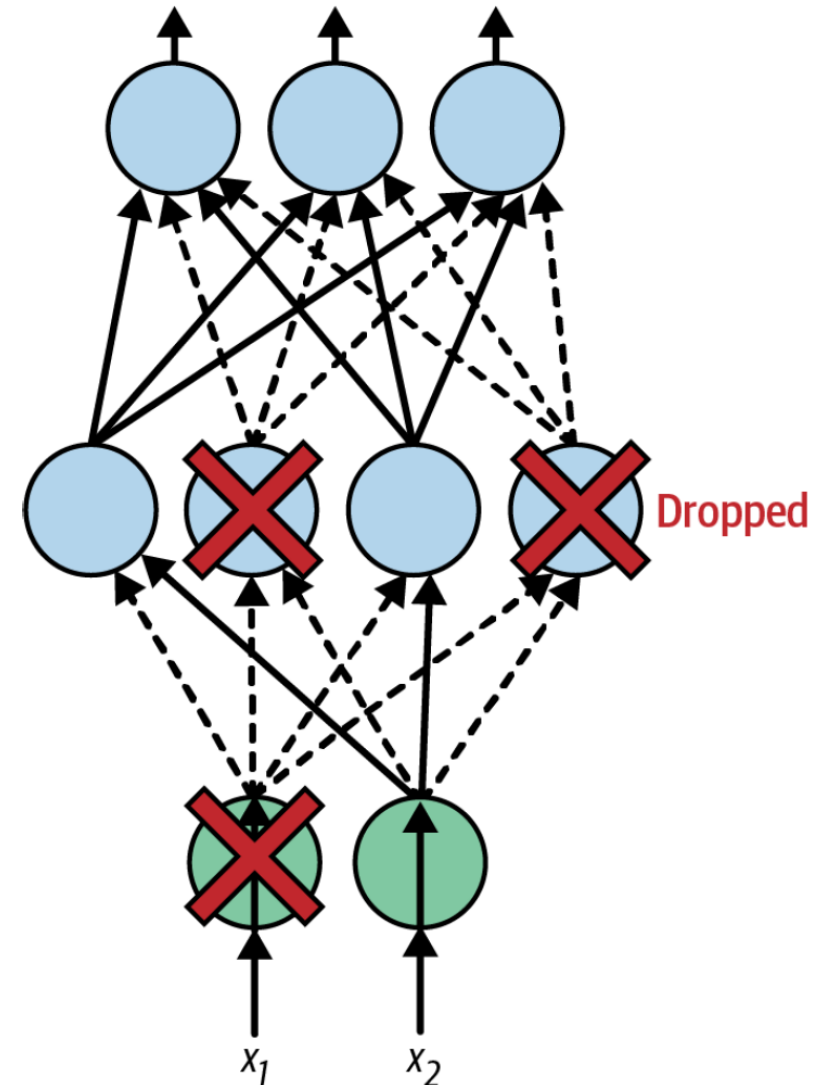
# Abandono (Drop-out)

- Al principio sorprende que esta técnica destructiva funcione.
- ¿Funcionaría mejor una empresa si sus empleados tuvieran que lanzar una moneda al aire cada mañana para decidir si van o no a trabajar? Bueno, quién sabe; ¡quizá sí!
- La empresa se vería obligada a adaptar su organización; no podría confiar en una sola persona para hacer funcionar la máquina de café o realizar cualquier otra tarea crítica, por lo que estos conocimientos tendrían que repartirse entre varias personas.
- Los empleados tendrían que aprender a cooperar con muchos de sus compañeros, no sólo con un puñado de ellos. La empresa sería mucho más resistente. Si una persona renunciara, no habría mucha diferencia. No está claro si esta idea funcionaría realmente para las empresas, pero desde luego sí para las redes neuronales.
- Las neuronas entrenadas con abandono no pueden coadaptarse con sus neuronas vecinas; tienen que ser lo más útiles posible por sí solas. Tampoco pueden depender excesivamente de unas pocas neuronas de entrada; deben prestar atención a cada una de sus neuronas de entrada. Acaban siendo menos sensibles a los pequeños cambios en las entradas. Al final, se obtiene una red más robusta que generaliza mejor.



# Abandono (Drop-out)

- Otra forma de entender el poder del abandono es darse cuenta de que en cada paso del entrenamiento se genera una red neuronal única.
- Dado que cada neurona puede estar presente o ausente, hay un total de  $2^N$  redes posibles (donde  $N$  es el número total de neuronas droppables).
- Se trata de un número tan grande que es prácticamente imposible que la misma red neuronal se muestree dos veces. Una vez que se han ejecutado 10.000 pasos de entrenamiento, se han entrenado 10.000 redes neuronales diferentes, cada una con una sola instancia de entrenamiento.
- Evidentemente, estas redes neuronales no son independientes porque comparten muchos de sus pesos, pero aun así son todas diferentes. La red neuronal resultante puede considerarse un conjunto promedio de todas estas redes neuronales más pequeñas.



# Abandono (Drop-out)

- Hay un pequeño pero importante detalle técnico. Supongamos que  $p = 75\%$ : por término medio, sólo el 25% de todas las neuronas están activas en cada paso durante el entrenamiento. Esto significa que, después del entrenamiento, una neurona estaría conectada a cuatro veces más neuronas de entrada que durante el entrenamiento. Para compensar este hecho, tenemos que multiplicar por cuatro los pesos de conexión de entrada de cada neurona durante el entrenamiento. Si no lo hacemos, la red neuronal no funcionará bien, ya que verá datos diferentes durante y después del entrenamiento. En términos más generales, tenemos que dividir los pesos de conexión por la probabilidad de mantenimiento ( $1 - p$ ) durante el entrenamiento.
- En la práctica, normalmente sólo se debe aplicar dropout a las neuronas de las tres capas superiores (excluida la capa de salida).

# Monte Carlo(MC) Drop-out

- En 2016, un artículo de Yarin Gal y Zoubin Ghahramani añadió algunas buenas razones más para utilizar dropout.
- En primer lugar, el artículo establecía una profunda conexión entre las redes dropout (es decir, redes neuronales que contienen capas dropout) y la inferencia bayesiana aproximada, (Deep Gaussian Process) dando al dropout una sólida justificación matemática.
- En segundo lugar, los autores introdujeron una poderosa técnica llamada MC dropout, que puede aumentar el rendimiento de cualquier modelo dropout entrenado sin tener que volver a entrenarlo o incluso modificarlo en absoluto. También proporciona una medida mucho mejor de la incertidumbre del modelo y puede aplicarse con unas pocas líneas de código.
- MC Dropout amplía la técnica de dropout estándar utilizada durante el entrenamiento aplicando también el dropout en el momento del test y promediando las predicciones de múltiples pases hacia delante. Esto contrasta con el dropout estándar, que normalmente sólo se utiliza durante el entrenamiento para evitar el sobreajuste. MC Dropout utiliza esencialmente el dropout para realizar una aproximación bayesiana, lo que permite una mejor estimación de la incertidumbre en las predicciones de la red neuronal.

# Regularización Max-Norm

- Para cada neurona, restringe los pesos  $\mathbf{w}$  de las conexiones entrantes de tal forma que  $\|\mathbf{w}\|_2 \leq r$ , donde  $r$  es el hiperparámetro max-norm y  $\|\cdot\|_2$  es la norma  $\ell_2$ .
- La regularización Max-norm no añade un término de pérdida de regularización a la función de pérdida global. En su lugar, se suele implementar calculando  $\|\mathbf{w}\|_2$  después de cada paso de entrenamiento y reescalando  $\mathbf{w}$  si es necesario ( $\mathbf{w} \leftarrow \mathbf{w} r / \|\mathbf{w}\|_2$ ).
- Reducir  $r$  aumenta la cantidad de regularización y ayuda a reducir el sobreajuste. La regularización Max-norm también puede ayudar a aliviar los problemas de gradientes inestables (si no está utilizando la normalización por lotes).

Código

# Resumen y orientaciones prácticas

- Es posible que encuentres una configuración que funcione bien en la mayoría de los casos, sin necesidad de ajustar mucho los hiperparámetros. Dicho esto, ¡no consideres estos valores por defecto como reglas rígidas!
- Si la red es una simple pila de capas densas, entonces puede auto-normalizarse, y deberías usar esta configuración en su lugar.

| Hyperparameter         | Default value                           |
|------------------------|-----------------------------------------|
| Kernel initializer     | He initialization                       |
| Activation function    | ReLU if shallow; Swish if deep          |
| Normalization          | None if shallow; batch norm if deep     |
| Regularization         | Early stopping; weight decay if needed  |
| Optimizer              | Nesterov accelerated gradients or AdamW |
| Learning rate schedule | Performance scheduling or 1cycle        |

| Hyperparameter         | Default value                    |
|------------------------|----------------------------------|
| Kernel initializer     | LeCun initialization             |
| Activation function    | SELU                             |
| Normalization          | None (self-normalization)        |
| Regularization         | Alpha dropout if needed          |
| Optimizer              | Nesterov accelerated gradients   |
| Learning rate schedule | Performance scheduling or 1cycle |

# Resumen y orientaciones prácticas

- No olvides normalizar las características de entrada.
- También deberías intentar reutilizar partes de una red neuronal preentrenada si puedes encontrar una que resuelva un problema similar, o utilizar preentrenamiento no supervisado si tienes muchos datos sin etiquetar, o utilizar preentrenamiento en una tarea auxiliar si tienes muchos datos etiquetados para una tarea similar.
- Aunque las directrices anteriores deberían cubrir la mayoría de los casos, he aquí algunas excepciones:
  - Si necesita un modelo disperso, puede utilizar la regularización  $\ell_1$  (y, opcionalmente, poner a cero los pesos pequeños después del entrenamiento). Si necesitas un modelo aún más disperso, puedes utilizar el kit de herramientas de optimización de modelos de TensorFlow. Esto romperá la auto-normalización, por lo que debe utilizar la configuración por defecto en este caso.
  - Si necesita un modelo de baja latencia (que realice predicciones a la velocidad del rayo), puede que tenga que utilizar menos capas, utilizar una función de activación rápida como ReLU o ReLU con fugas, y plegar las capas de normalización de lotes en las capas anteriores después del entrenamiento. Tener un modelo disperso también ayudará. Por último, puede que quieras reducir la precisión de flotación de 32 bits a 16 o incluso 8 bits.
  - Si está construyendo una aplicación sensible al riesgo, o la latencia de inferencia no es muy importante en su aplicación, puede utilizar MC dropout para aumentar el rendimiento y obtener estimaciones de probabilidad más fiables, junto con estimaciones de incertidumbre.